

Exposé: Implementierung eines In-Memory Key-Value Stores auf Basis von optimierten B⁺-Bäumen

Janne Gruhler, Maxim Bellm, Albert Rico Martin, Matthias Raba

Department of Computer Science – Hochschule Furtwangen – Furtwangen, Germany

I. EINFÜHRUNG UND MOTIVATION

B⁺-Bäume zählen zu den grundlegendsten Indexstrukturen in Datenbanksystemen, galten für In-Memory-Szenarien aber lange als unterlegen: Systeme wie Redis setzen auf hash-basierte Strukturen mit nahezu konstanter Latenz pro Zugriff. Das Paper “*B-trees are back: Engineering fast and pageable node layouts*” von Müller, Benson und Leis [1] zeigt, dass dies vor allem implementierungsbedingt ist. Durch mehrere Knotentypen, Fingerprinting, Hinting, Präfixkompression und SIMD-Beschleunigung erreichen B⁺-Bäume vergleichbaren Durchsatz wie der Adaptive Radix Tree (ART) und der Height Optimized Trie (HOT). Ziel dieses Projekts ist es, diese Ergebnisse nachzuvollziehen und einen eigenen, Redis-kompatiblen Key-Value Store auf dieser Basis zu implementieren.

II. ANALYSE UND MODERNISIERUNG DES REPOSITORIES

Der Code des Papers ist öffentlich auf GitHub verfügbar, war aber stark auf eine spezifische Compilerumgebung zugeschnitten. SIMD-Intrinsics waren direkt als AVX2-Code eingebettet und das Build-System bestand aus einem handgeschriebenen, monolithischen Makefile. Wir haben das Repository in eine statische Bibliothek (`libbtree`) umgebaut. Dazu haben wir das Build-System auf CMake migriert, die SIMD-Logik hinter plattformunabhängigen Abstraktionen gekapselt und den Code auf modernes C++ aktualisiert. Eine einheitliche `DataStructureWrapper`-Schnittstelle macht die Bibliothek einfach in andere Projekte einbindbar.

III. NACHVOLLZIEHEN DER MESSERGEBNISSE

Die Benchmarks des Papers laufen über Python-Skripte, die Benchmarkingprogramme mit Parametern wie Schlüsselanzahl, Operationsanzahl und Batchgröße aufrufen, Ergebnisse als CSV speichern und in R visualisieren. Als Datensätze kamen dichte und dünn besetzte Integer-Schlüssel, Wikipedia-Artikeltitel und zufällig generierte URLs zum Einsatz. Das Nachvollziehen war aufwändig: Die verwendeten Datensätze sind nur im Paper beschrieben, nicht im Repository beigelegt. Wir mussten sie selbst generieren, was die Vergleichbarkeit einschränkt. Nicht alle Grafiken ließen sich eindeutig einem Skript zuordnen, und unsere Messwerte wichen teils um Größenordnungen von den publizierten Werten ab. Nach Kontaktaufnahme mit einem der Autoren bekamen wir Zugriff auf die Originaldatensätze. Der Autor bestätigte, dass einige Messungen unter schwer reproduzierbaren Bedingungen entstanden sind.

IV. IMPLEMENTIERUNG EINES REDIS-KOMPATIBLEN KEY-VALUE STORES

Auf Basis von `libbtree` haben wir in C++ einen Key-Value Store gebaut, der das Redis Serialization Protocol (RESP) spricht und damit als Drop-in-Ersatz für Redis eingesetzt werden kann. Die Netzwerkschicht basiert auf Boost.Asio. Unterstützt werden die gängigen Kommandos GET, SET, MGET, MSET, DEL, EXISTS, INCR/DECR und INFO. Zum Vergleich haben wir YCSB2 genutzt, das eine identische Last gegen beide Server richtet. Verwendet wurde Workload C (100 % Lesezugriffe, Zipf-verteilt) über Batch-MGET-Anfragen. Abb. 1 zeigt den Durchsatzvergleich nach Datenbankgröße. Ab ca. 500 000 Einträgen ist unser System kompetitiv. Bei kleineren Datenmengen überwiegt der Aufbauoverhead des Baums gegenüber Redis’s $O(1)$ -Hash-Lookups. Ein struktureller Vorteil von B⁺-Bäumen ist die geordnete Schlüsselspeicherung, die Range-Scans ohne zusätzliche Datenstrukturen ermöglicht. Bei Redis ist das nicht möglich, da Hashing grundsätzlich keine Ordnung beibehält.

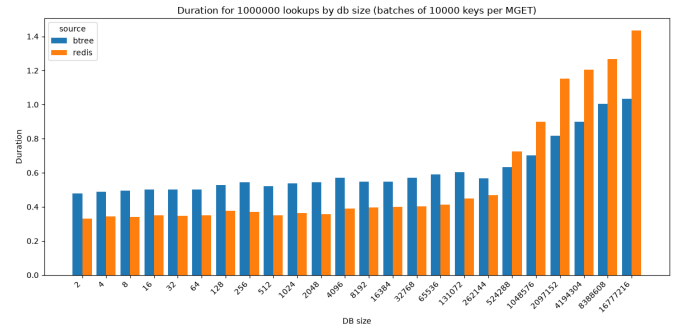


Fig. 1: YCSB2 Workload C: Durchsatz btree-redis vs. Redis nach Datenbankgröße.

V. FAZIT

Die im Paper beschriebenen optimierten B⁺-Bäume sind neben disk-basierten Datenbanken eine solide Grundlage für wettbewerbsfähige In-Memory Key-Value Stores. Die Messergebnisse des Papers konnten nur teilweise reproduziert werden, was zeigt, wie wichtig eine saubere experimentelle Dokumentation in der Systemforschung ist.

REFERENCES

- [1] M. Müller, L. Benson, and V. Leis, “B-trees are back: Engineering fast and pageable node layouts,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–26, 2025.